DREW

Lecture 32: Pointers

CSCI 260 Computer Systems & Architecture
Fall 2024

Andy Siyuan Feng

11/15/2024

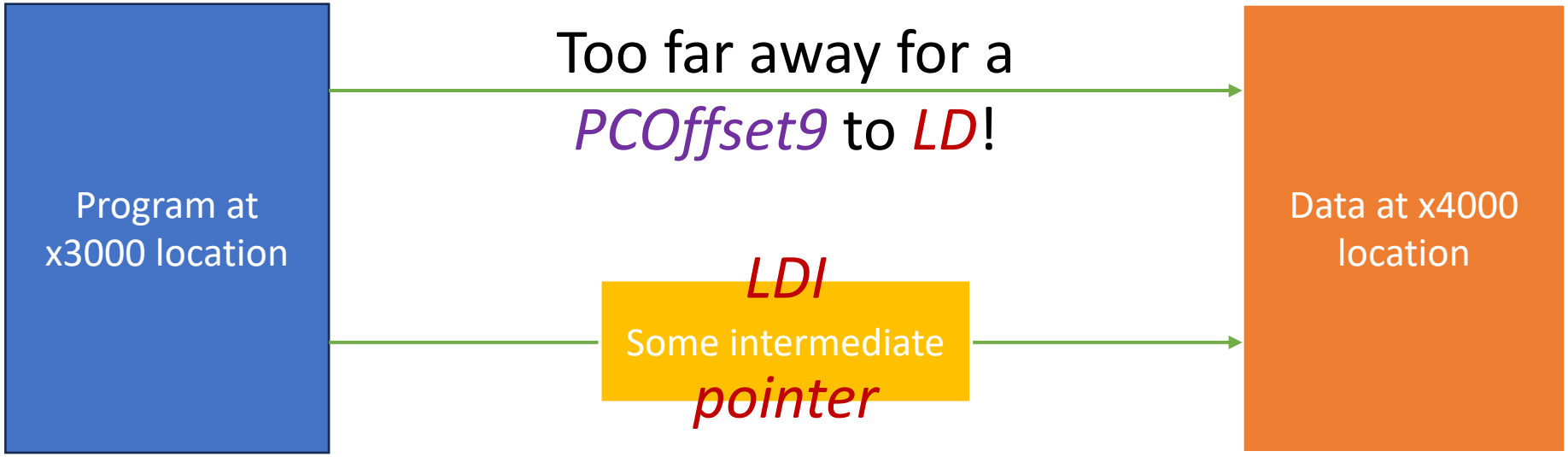
The Terms

- **Recall**: Can define a string either as a *char array*:
  - char arr[20]
  - or as
  - char* str, i.e., a string

- **Def.**: A *Pointer* is the memory address of a variable!
- Pointers allow us to access variables INDIRECTLY. Recall LC-3.

Address v.s. Value of a Variable

- Sometimes, want to deal with address of memory locations; recall LC-3.



- When adding 10 numbers in the memory address, used R2 as the tracker for which number to add.
- R2, in this case, is a pointer; it points towards the memory address of the data we are interested in.

Address v.s. Value of a Variable

- Another aspect of why pointers: has something to do with scope and return values.

- **Q**: How do we swap 2 integer values using a function?
  1. The swapped values are not reflected in the caller function due to scope.
  2. Meanwhile, cannot return 2 values.

```
void swap (int num1, int num2) {
        int temp = num1;
        num1 = num2;
        num2 = temp;
}
```

NOT working.

Address v.s. Value of a Variable

- What happened in memory for *swap(int, int)*?

- **Observation**: *swap(int, int)* requires addresses of variables in main to assign back the swapped values.

- What are addresses in C?

  Pointers

Pointers in C

- In C, a pointer is also a variable.
- Takes a data type and makes it a pointer; i.e., the pointer is pointing towards a data that's of that data type.

- To **declare** a pointer: We use * to make a variable a pointer.

```
// dataType*  varName   or   dataType  *varName

int* p;         // p is a pointer pointing towards an int
float* q;       // q is a pointer pointing towards a floating-point
char* ch;       // ch is a pointer pointing towards a char
```

Pointers in C

- Two operations on pointer:

1. *p*: returns the value pointer *p* is pointing towards.
2. *&var*: returns the address of variable *var*.

- **Ex.**:

```
int obj = 4;
int *ptr;

ptr = &obj;

*ptr = *ptr +1;
```

ptr is initialized with a random address.

Location identified by ptr is now stored with address of obj.

Take value pointed by ptr and add 1, assign the sum to location id.ed by ptr. Since ptr is the address of obj, obj's value is updated.

DREW

Pointers Example – 1

- Let's debug the program.

```
int main() {
    int obj = 4;   obj: 4
    int* ptr;   ptr: 0x27

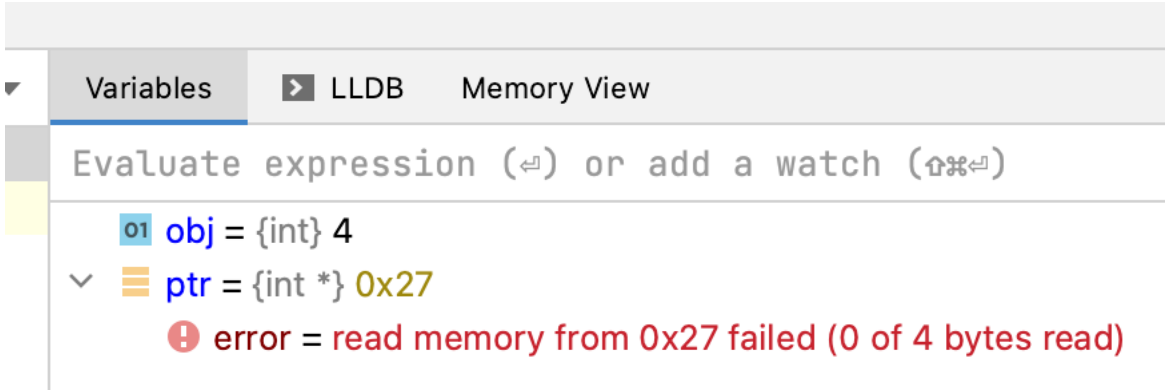
    ptr = &obj;   obj: 4    ptr: 0x27
```

- **Ex.:**

```
int obj = 4;
int *ptr;

ptr = &obj;
long temp1 = ptr;
int temp2 = *ptr;
int temp3 = *ptr + 1;

*ptr = *ptr +1;
```



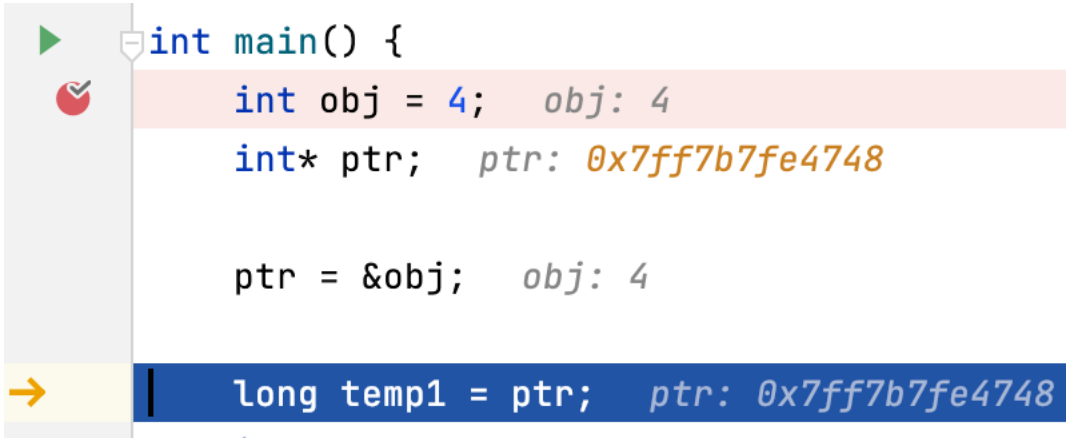
Pointers Example – 2

- Let's debug the program.
- **Ex.:**

```
int obj = 4;
int *ptr;

ptr = &obj;
long temp1 = ptr;
int temp2 = *ptr;
int temp3 = *ptr + 1;

*ptr = *ptr +1;
```





DREW

Pointers Example – 3

- Let's debug the program.
- **Ex.:**

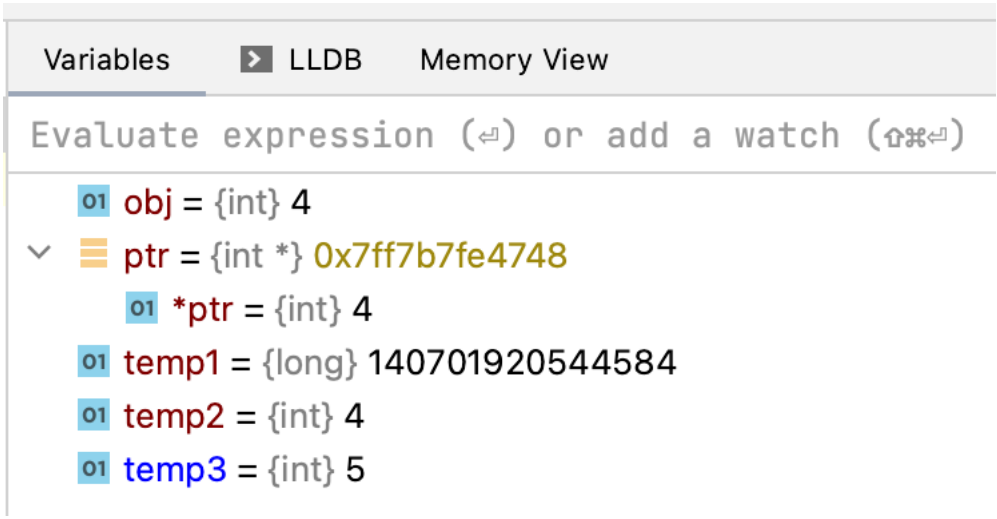
```
int obj = 4;
int *ptr;

ptr = &obj;
long temp1 = ptr;
int temp2 = *ptr;
int temp3 = *ptr + 1;

*ptr = *ptr +1;
obj++;
```

```
long temp1 = ptr;   temp1: 140701920544584
int temp2 = *ptr;   temp2: 4
int temp3 = *ptr + 1;   temp3: 5

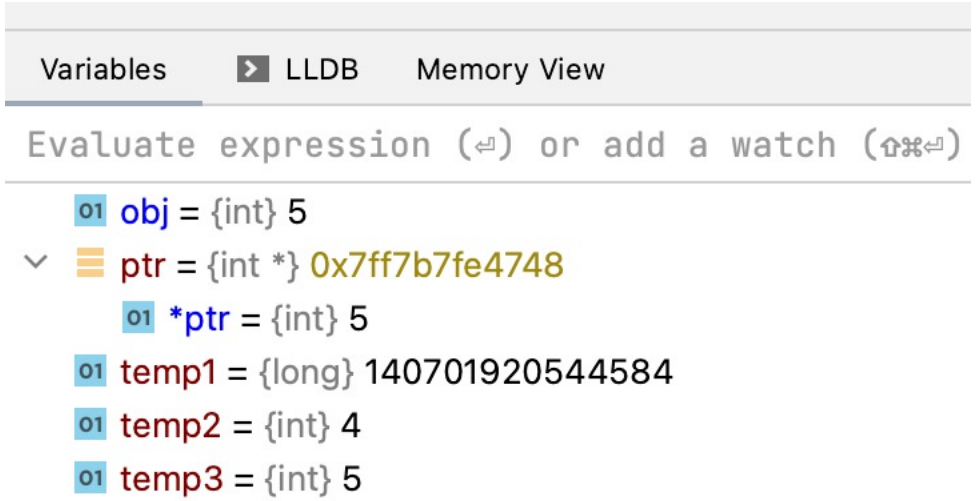
*ptr = *ptr+1;   ptr: 0x7ff7b7fe4748
```

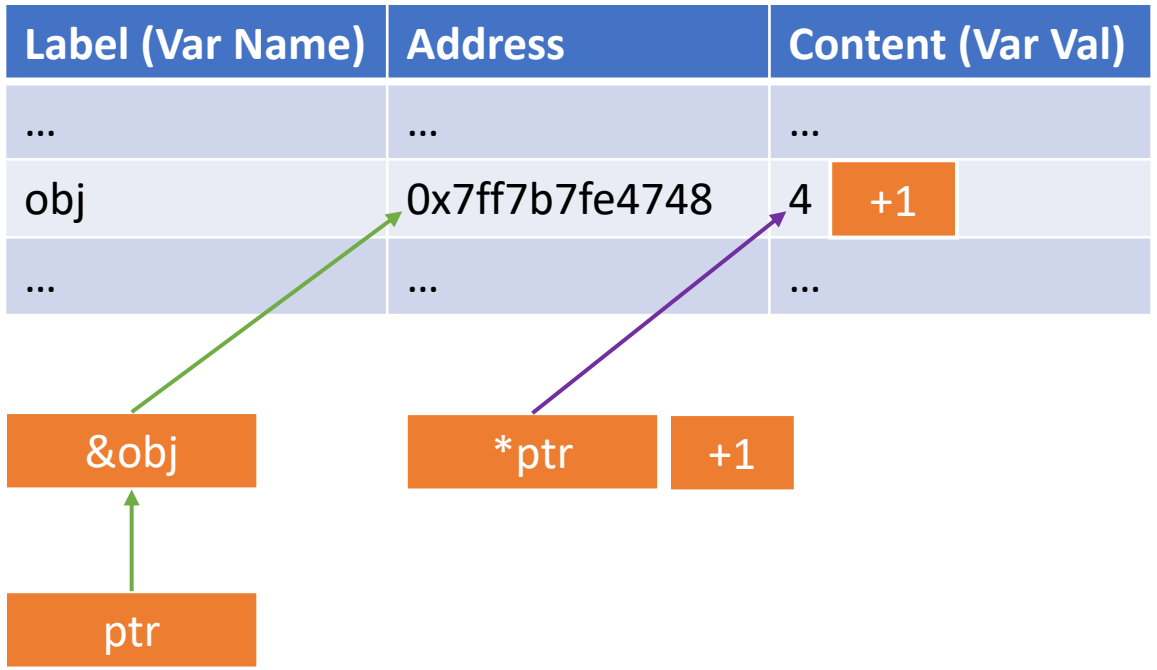


Pointers Example – 4

- Let's debug the program.
- **Ex.:** … …

```
*ptr = *ptr +1;
obj++;
return 0;
```



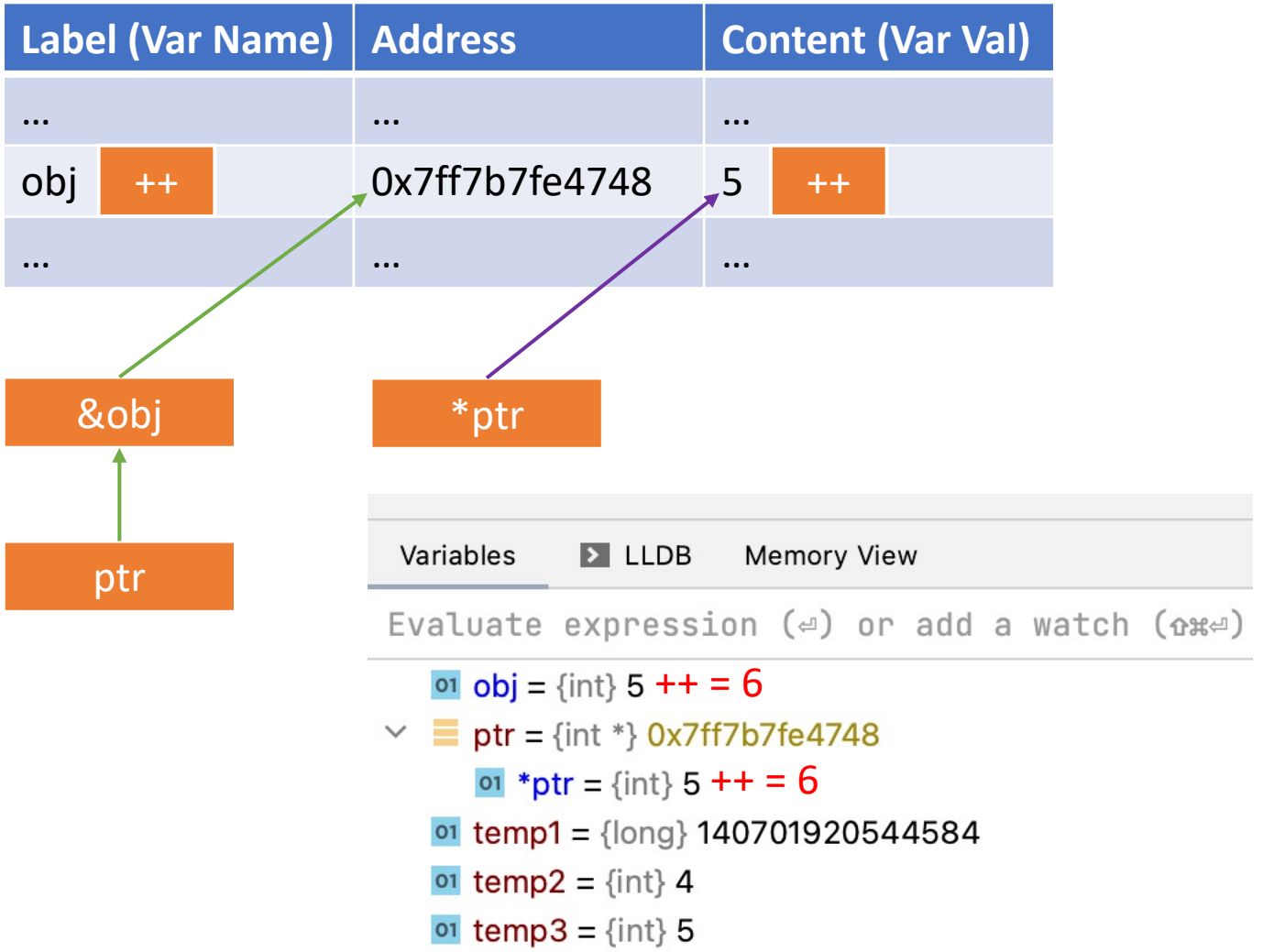


Pointers Example – 4

- Let's debug the program.
- **Ex.:** … …

```
*ptr = *ptr +1;
obj++;
```

→ `return 0;`



Passing by Reference – Pointers as Arguments

- Since pointer is a variable, it can be passed as an argument into a function. Known as *Passing by Reference*.
  - If *Passing by Value*, then changes in the function won't be reflected in the caller scope (and may be hard to return).
  - *Passing by Reference:* Passing address via pointer, and changes on pointer content will be reflected in the referred variable.



Passing by Reference – Pointers as Arguments

- Recall the swap example, here's what we can do; on the **caller side**:

```
int num1 = 3;
int num2 = 10;

int* num1_ptr = &num1;
int* num2_ptr = &num2;

swap_ptr(num1_ptr, num2_ptr);
```

OR, simply

```
swap_ptr(&num1, &num2);
```

DREW

Passing by Reference – Pointers as Arguments

- Recall the swap example, here's what we can do; on the **callee side**:

```
void swap_ptr (int* num1_ptr, int* num2_ptr) {

        int temp = *num1_ptr;    // temp gets val pointed by num1_ptr

        *num1_ptr = *num2_ptr;          // content overwriting

        *num2_ptr = temp;               // content overwriting
}
```

Passing by Reference – Pointers as Arguments

- Now, debug in IDE

```c
#include <stdio.h>

void swap_ptr(int* num1_ptr, int* num2_ptr) {
   int temp = *num1_ptr;
   *num1_ptr = *num2_ptr;
   *num2_ptr = temp;

   return ;
}

int main() {
   int num1 = 3;
   int num2 = 10;
//   swap_ptr(&num1, &num2);

   int* ptr1 = &num1;
   int* ptr2 = &num2;
   swap_ptr(ptr1, ptr2);

   return 0;
}
```

```c
void swap_ptr(int* num1_ptr, int* num2_ptr) {   num1_ptr: 0x7ff7b51ae748   num2_ptr: 0x7ff7b51ae744
    int temp = *num1_ptr;   temp: 3
    *num1_ptr = *num2_ptr;   num1_ptr: 0x7ff7b51ae748   num2_ptr: 0x7ff7b51ae744
    *num2_ptr = temp;

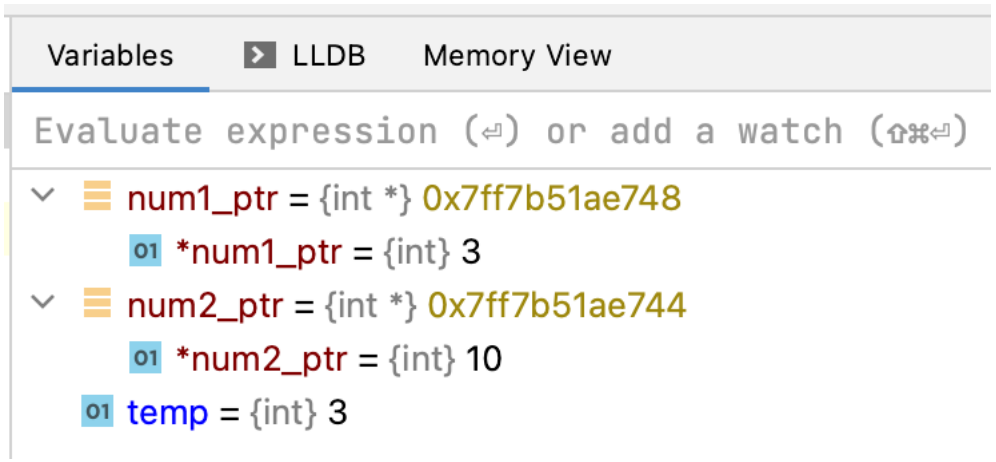
    return ;
}

int main() {
    int num1 = 3;
    int num2 = 10;
//    swap_ptr(&num1, &num2);

    int* ptr1 = &num1;
    int* ptr2 = &num2;
    swap_ptr( num1_ptr: ptr1,  num2_ptr: ptr2);

    return 0;
}
```

Stepped Into here.



DREW